

FitTrack 社交健身系统: 推演到机器级,然后把它砍到最简

实时足迹 feed · 个人 Top Activities · Leaderboard —— 三表极简架构完整推演 (V2)

方法论:Features → Data Flow → Scale → Failure → KPI,两遍法展开

一 · 题面与规模锚点

设计社交健身应用 FitTrack, 三项需求: R1 用户运动时 follower 近实时看到进度(位置/配速/累计里程); R2 按某天/某月/过去一年查个人 top activities; R3 每周里程排行(全局榜 + 好友榜)。

规模: 5000 万注册 / 500 万 DAU / 峰值 100 万并发 workout, 手机每 5 秒上报一个遥测点; 事件会迟到、乱序、重复; 存在百万粉丝的名人账户。

两行规模锚点(所有数字的口径来源):

① 题面: 50M 注册 / 5M DAU / 1M 并发; ② 容量按 1B 用户上界设计(约 200 倍余量)。两套账切换前先声明口径。
自设参数: feed 刷新闻隔 10s; 好友上限 1000。

二 · 方法论: 分层瀑布 × 两遍法

- 分层瀑布: Features → Data Flow → QPS/Scale → Failure → KPI——先定契约, 再画流, 再上数字, 最后谈死法;
- 两遍法: 第一遍用几分钟把端到端数据流横扫一遍(三个需求的出口全部指到, 完成度先锁死); 第二遍逐个组件回填容量数字与故障处理。就算第二遍被打断, 第一遍已保住全覆盖——完成度是这类设计的硬评分轴;
- 本文还记录了推演中被砍掉的组件(第九章)——设计的过程, 一半是搭, 一半是砍。

三 · Features(功能契约)

- 两层 UI: newsfeed 列表页(每个在跑好友一条 snippet: 在跑?/时长/里程/街区级粗位置) → 点击进 detail 页(完整轨迹、配速)。拆分理由: 载荷与 QPS 分级;
- 在跑才显示: 判活由数据驱动——遥测持续到达即在跑, 停更 20s 自动过期(TTL 语义), 不依赖 end 事件善后;
- 隐私: 位置只到街区/公园粒度 + 用户开关; 轨迹只保留最近 ~10 分钟(见第六章, 保留策略即隐私窗);
- viewer 断网: 客户端缓存最后已知状态, 标注「更新于 X 分钟前」;
- 地理编码: 客户端本地解析(粗粒度、零服务器成本)为主; 服务器端距离阈值 + geohash 网格缓存为备;
- 点赞/评论列为 v2。

四 · 第一遍: 端到端数据流(一根脊柱, 三个出口)

client 每 5s 上报 {userid, eventid, ts, gps, 街区id, type} (~50B; eventid = 每次跑步唯一, userid+随机串; 断网重连: 同 eventid 增量拼接, 异则整体刷新)
→ 接入/日志层(append 落盘, ts 为 key, 真相源; 收、存、处理一体)
→ 更新流三路分发:
① Snippet 表(userid → 最新态) = R1 列表页出口;
② Tracking 表(userid → 最近 100 点滑动窗) = R1 detail 页出口;
③ 离线派生: activity 摘要行(每次跑步收敛一行) = R2 出口; 摘要滚周累计 = R3 出口(global batch top100 / 好友 multiget 现排)。
三个需求全部落位——完成度锁死, 第二遍开始回填。

五 · 第二遍: 件1: 接入/日志层(4-6 台, 收存管一体)

- 写入 QPS: $1M \text{ 并发} \div 5s = 200K \text{ events/s}$ 峰值(口诀: $QPS = \text{并发数} \div \text{上报间隔}$); 均值 1-2 万/s, 峰均比 ~10:1;

- 带宽: $200K \times 50B = 10MB/s$ ——单机吞吐即可;分 2-3 片 $\times$ 主从的理由不是吞吐,是单点故障半径 + 下游并行;带本地盘防断电;
- 存量:3 个月原始日志 $\approx 78TB$ (压缩 $\sim 1/5$),十年 $< 1PB$ ——raw 必须留:debug / 恢复 / 离线统计的唯一真相源,一切派生可重算;
- 涨落:相位独立时某秒到达 $\approx 200K \pm 400(\pm 0.2\%, \text{Binomial})$;独立性由 jitter 强制成立(固定周期 + 随机相位,均值不变);
- 风暴处置:区域断网重连时瞬时到达可超 200K——日志层吸收成积压,消费端按均值 $\times 2-3$ 配速排空(排队余量);演算:断网 1 分钟积压 ~ 1200 万条,2 倍速率 30 秒排空——「分钟级风暴,秒级消化」;
- 故障:副本顶替 + 按 offset 重放,(eventid, ts) 幂等 key 保证重放安全。

六 · 件2:Snippet 表——全量复制,零 fan-out(本设计的核心)

6.1 结构与容量

- 每用户一行最新态:在跑状态位(1B)+ 街区 id(8B)+ 速度(1B)+ 最新更新时间(8B)+ 最新 eventid(8B) $\approx 28B$,压缩 $\sim 5B$;
- $28B \times 1B$ 用户 = 28GB——单机内存放得下 $\rightarrow$ 全量复制 ~ 20 台,每台一份完整副本;
- presence 溶解:「在不在跑」就是表里的一个状态位(由「最新更新时间 + 20s 窗口」判定)——不需要独立的 presence 服务;一次本机 multiget 同时完成「谁在跑 + snippet 内容」,两步读合成一步。

6.2 读写账与灵魂论证

- 读: $5M \times (7200s \text{ 在线} \div 10s \text{ 刷新}) = 3.6B \text{ 次/天} \div 86400 \approx 4 \text{ 万 QPS 均值} \rightarrow \times 10 \text{ 包络} = 40 \text{ 万峰} \rightarrow$ 每台 2 万;好友列表(≤ 1000 id)在本机哈希表一把查完,50-100 微秒,零跨机 fan-out;
- 写:每台副本自行订阅更新流、批量 apply 200K/s(副本 pull 型同步,无人做推送 fan-out);
- 灵魂论证:写放大 $\times 20$ 买掉读侧全部 fan-out——读 40 万 $>$ 写 20 万,这笔税交得划算。把读 heavy 换成可接受的写 heavy,是全量复制方案的本质。

6.3 尾部治理:cap 管尾部,期望值管配置

- 产品封顶:列表最多展示 20-100 个在跑好友(按开跑时间排序截断)——UI 本来放不下,产品截断 = 天然背压;
- 期望负载:在跑率 = $1M/50M = 2\%$,中位好友 300 $\rightarrow$ 平均 ~ 6 个在跑好友——载荷读极小;
- 最坏情形(全员满帽):40 万刷新 $\times cap 100 = 4000$ 万查/s $\div 140$ 台快照层 = 28 万/台——仍在单机内存查找能力内。上下两界皆封死:worst case 设保险丝,期望值定舰队。

七 · 件3:Tracking 表——分片,滑动窗即隐私窗

- 每用户最近 100 个轨迹点($\times 32B \approx 3.2KB$):5s 一个点 $\rightarrow$ 窗口 $\approx 8-10$ 分钟——「只保留最近 100 点」一句话同时解决存储有界与隐私上限;
- 容量:活跃口径 $1M \times 3.2KB \approx 3.2GB$;1B 全员上界 $3.2TB \div 20 \text{ 片} \approx 160GB/\text{台}$ ——放不进单机,所以分片;
- 写:200K 点/s $\div 20 \text{ 片} = 1 \text{ 万环形写/台}$ (定长环,新点顶旧点,微秒级);
- 读:detail 页点击才进,QPS 低一至两个量级;单 key(userid)直达分片——fan-out 只存在于「一次查多人」的路径,单 key 点查天然免疫;
- 更久的历史轨迹走 raw 归档的低频路径。

件2 与件3 的对照,即本设计的核心手筋:瘦表复制、胖表分片。小而全局要查的表(snippet)用全量复制买掉 fan-out;大而按 key 点查的表(tracking)用分片摊容量——各得其所,互不越位。

八· 件4:离线派生——R2 / R3

- Activity 摘要表:workout 结束(或 TTL 判定)时收敛一行 {距离/时长/日期/...};写频 = 结束率,峰值仅千级/s;key = userid 分区 + ts 片内排序 → 「取某人最近 100 次」= 单分片连续范围扫,毫秒内;
- R2(top activities):个人一年也就几百行——读时现算,rollup 预聚合在这个量级是过度设计(把这句 trade-off 说出口);随机插入由 LSM/memtable 吸收成顺序写;
- R3(leaderboard)双轨:global 榜每小时 batch job 算 top100 → cache 直读(预算侧);好友榜 client 带好友列表 → multiget 周累计 → 现排(1000 人 < 0.1ms,读时算侧)——增量预算 vs 读时重算的不对称,正是这两条查询负载画像不同的镜像;
- KPI 一句:分层 counter(QPS/延迟/积压/水位)+ Spark/Presto 离线聚合,与 DAU/点击等产品指标合看健康度。

九· 剃刀记录:被砍掉的五台机器

推演过程砍掉的组件,每一刀都有账——记录在此,因为它们是设计的另一半:

被砍组件	砍它的账
显式 dedup / seq 机制	读路径吃的是最新态快照,天然幂等;写路径把去重折进 (eventid, ts) 幂等 upsert 的 key 设计——机器消失,语义保留
事件驱动的「活跃好友名单」子系统	start/stop 扇出、per-viewer 名单、对账——四件新家当;而笨 multiget 打全量复制表的账本已经算得过来。留作演化预案:刷新压到秒级时再请它出山
Bloom filter 存在性过滤	活跃集 8MB-28GB,小到可以精确全量复制——概率结构的存在意义是「集合大到抄不动」,这里不成立;1% 误报反而白读载荷
独立 presence 服务	snippet 行加一个状态位 + 时间戳即等价——「瘦则合,胖则拆」;字段将来变胖再拆出去
per-viewer inbox 写扩散(朋友圈式)	本题遥测更新 200K/s × 粉丝数,push 判死刑;判决公式:push 成本 = 发布率 × 粉丝数,pull 成本 = 刷新 QPS × 关注数——低频发帖场景公式翻面,push 才复活

十· 演化预案(兜里的四张牌)

- 名人账户:本架构 pull 型读天然免疫百万粉丝——名人不产生任何特殊路径;若未来引入写扩散才需要粉丝阈值豁免;
- 朋友圈/post 变体:判决公式翻面(发帖 500/s × 普通好友数 → 写扩散可行),或走 pull + 时间槽发帖索引(10 分钟一槽 ~6MB,全量复制同款手筋);存储 (userid, ts, postid) 复合 key,分片按 userid、片内按 ts 排序;
- 写路径升级:接入层如需严格多消费者语义,自然演化为标准持久化日志(性质:落盘、可重放、多读者、按 user 保序);
- snippet 字段变胖:再把活跃位拆回独立小表——合与拆只是尺寸的函数。

十一· 判决口诀(全文的压缩)

口诀	一句话展开
QPS = 并发数 ÷ 上报间隔	「状态量」换算「流量」的唯一桥梁
单位纪律	每个数缀单位:/天 vs /秒;/台 vs /副本组——两族算错各灭一半
Sanity check	任何 QPS 对照「Google 全站 ~百万级」,超过了必是单位错
预算 = QPS × 扇出宽度 × 单价	fan-out 无罪,三因子乘积超标才是罪
热数据够小 → 全量复制	用写放大买掉读 fan-out;读 > 写时,这笔税划算
瘦表复制,胖表分片	小而全局查的复制;大而点查的分片
cap 管尾部,期望值管配置	worst case 设保险丝,不买机器
key 跟着查询长,表跟着分发策略长	schema 由访问模式反推
raw 是真相源,派生可重算	日志重放 + 幂等 key = 一切下游故障的退路
悲观界证可行性,真实值定预算	两个数都报,无懈可击
说性质不说品牌	「落盘、可重放、多读者、按 user 保序的日志」

收尾一句:在这个量级下,热数据小到可以全量复制——所以最简单的架构给出了最强的读路径。机器总账:端到端约 50-70 台吃下题面规模,1B 上界各层线性扩。

附篇·推演历程(从五表到三表的路)

终版之前,推演走过更长的路。以下三章按原样保留当时的机器级设计——它们不是废案,是终版每一刀的账本来源,其中 post 子系统更是完整的独立变体设计。

附A·Presence 独立服务版(后被溶解进 snippet 表)

- 表:userid → last_active_ts(8B)+ active 位(读写双 buffer,每 5s 整体换一版——与 base/delta 同手筋);更新源钉死:只有 workout 遥测续期,viewer 刷 feed 的请求绝不更新(否则全世界刷 feed 的人都会「跑起来」);
- 容量:题面口径 50M × ~10B ≈ 500MB;1B 上界 8-16GB——单机全量放得下 → 每台一份完整副本,20 台 = 20 副本,单请求零跨机 fan-out(本机哈希把 ≤1000 好友 id 全查完,50-100 微秒);
- 查询:40 万峰 ÷ 20 台 = 2 万/台;client 每 5s 携好友列表(8KB)→ 网卡账 3.2GB/s ÷ 20 = 160MB/s/台,单机 NIC 能力内;
- 写:更新流广播 × 20 副本(每台 320KB/s——写放大主动声明);
- 溶解理由:snippet 迷你行(28B)本身已含状态位 + 更新时间——同样全量复制的前提下,单独的 presence 表成了冗余;「瘦则合,胖则拆」,字段将来变胖再拆回来。

附B·变体推演:Post 信息流(朋友圈化)——完整机器级

判决公式:push 成本 = 发帖率 × 粉丝数;pull 成本 = 刷新 QPS × 关注数。遥测场景 200K 更新/s × 粉丝 → push 判死刑;朋友圈 5M × ~10 帖/天 ≈ 500 写/s(峰值千级)→ push 复活。同一堆零件,负载画像翻面,判决翻面。

- 定案路线:pull + 时间槽发帖索引——(userid, 发帖 ts) 全量副本小索引,10 分钟一槽:5000 万帖/天 ÷ 144 槽 ≈ 35 万条 × 16B ≈ 5-6MB/槽、~800MB/天——presence 同款手筋:小到整体复制;
- 分频轮询:snippet 每 5s、post 每 60s(per-viewer 好友发帖率 ~0.1 QPS)→ 读 3-30K QPS;
- 回溯:client 逐槽向前问「这槽里谁发过帖」→ 两步读;300K QPS 在已验的 400K 能力内;槽边界天然就是翻页游标;深翻旧账退化为按好友情性拉取;
- 存储两张皮:帖子表 (userid, ts, postid) 复合 key——分片只按 userid(同人同片),ts+postid 片内排序,翻页顺序扫不散落;内容 ~1KB 压缩 200B,like/comment 只存 id、detail 层分层读;容量 1B 上界 ×10 帖 ×200B×10 年 ≈ PB 级,实际 5M DAU 小 ~200 倍;
- 名人豁免:粉丝 > 1 万走 outbox-pull——病因不是发得多,是单帖爆炸半径(一帖 × 百万粉 = 百万级追加脉冲;堵塞 worker 或限速迟到,两头不划算);
- 「一个人发帖不多」的三个红利:outbox 微小可整存 cache;inbox 增长有界(~7KB/天/人,保留期收口);写侧无热点作者;
- 对照方案(inbox 写扩散)也算过账:fan-out worker 按目标分片攒批,30 万追加/s ÷ 30 分片 = 1 万/s/分片,全套 ~65 台——留作刷新压到秒级时的演化预案;
- 存储与分发正交:内容表是唯一事实;inbox/outbox 不是「数据」,是分发判决的物化——「表跟着分发策略长」,丢了可从内容表 + 关注图重建。

附C·写侧存储与处理的详账

- 原始日志三个月 ≈ 78TB(未压缩 10MB/s;压缩 ~1/5);若三个月全驻内存 = 140 台 × 100GB × 3 副本 = 420 台,买「任意历史秒级可查」——trade-off 说出口它是选择;退防线 = 近 7 天内存 + SSD 分层(~30-40 台);
- 分片按 (userid + eventid) hash 防倾斜;归档按接收时间 slot,serving 按 (eventid, client_ts) 索引——两个序各司其职;

- 内存 KV 随机写按 10K/s/台保守地板(实测 10 万+/s),append-only 更高——两个口径分开声明;
- 流处理悲观界:10ms/条 × 32 核 = 3.2K/s/台 → 60 台上界;真实微秒级 → 个位数台——「悲观界证可行性,真实值定预算」;
- user table 可用 base/delta 双份 + serve 指针切换(快照隔离,代价双倍内存)。

附D · Failure Modes 汇总表

故障	处置	依据
区域断网 → 重连风暴	日志吸收;消费按均值 ×2-3 排空	1 分钟积压 ~1200 万条,2 倍速率 30 秒排空
流处理器宕机/算错	按 offset 重放;(eventid, ts) 幂等	raw 为真相源,派生可重算
snippet 副本宕机	20 个全量副本任意顶替	秒级陈旧在 20s 判活窗下无感
接入层断电	本地盘落盘 + 冗余机	写路径不丢数据
跑者手机断网/没电	本地攒批补传;eventid 拼接规则;状态 last-write-wins(按 client_ts)防里程回跳;end 缺失 → TTL 兜底	既有机制,零新件
viewer 断网	缓存 + 「更新于 X 分钟前」;恢复先拉状态再拉 feed	诚实降级

全文收束:终版三表(日志真相源 / snippet 全量复制 / tracking 分片)+ 离线派生,端到端 ~50-70 台;附篇是每一刀背后的账。在这个量级下,热数据小到可以全量复制——最简单的架构给出最强的读路径。